

Fast Realistic Renering: Physically Based Rendering

Lluna Clavera Comas¹

¹Universitat Politècnica de Catalunya - BarcelonaTech

Imanol Munoz-Pandiella

In this project we have experimented with different implementations of physically based rendering methods to render different objects taking into account their materials. We have experimented with methods based on a point light and methods based on Image Based Rendering (IBL) and we developed an application to test all the explored methods. In this report we provide a detailed explanation of the used techniques along with a discussion of the results we obtained.

Keywords— $\LaTeX$ class, lab report, academic paper, tau-book class

1. Introduction

In this project we explore different techniques for physically based rendering using OpenGL and GLSL vertex and fragment shaders. The application we created (based on a skeleton provided by Imanol Munoz-Pandiella) supports loading different models, loading environment maps, changing the properties of the material (albedo, roughness and metalness) and even loading textures to specify said material properties.

In this application, we implemented five different shaders to render the given object: Using phong shading and a solid color, using textures, using a simple reflection of the environment map, using a simple PBR (Physically Based Rendering) shader and, finally, a PBR shader using Image Based Lighting (IBL). In this report we will explain the different techniques used for every shader. Additionally, we also implemented methods to precompute the diffuse irradiance map and prefiltered maps for the computation of the IBL PBR lighting. Finally, a button to extract and save the diffuse maps is also provided in the application.

2. Physically Based Rendering Techniques

In this section, we are going to explain the multiple methods that we used without going into much detail about the shader implementation (the shader implementation will be explained in the next section).

2.1. Phong Shading

Phong shading is one of the simplest lighting models out there (if we do not count just setting a solid color without lighting). Phong shading uses the normal vector of the surface, the view vector and the light vector to compute the illumination of the surface. This technique also uses the reflection of the light vector along the normal direction, but this new vector can be computed from the other three.

This model takes into account four different components to generate the lighting: The emitted light (which will be 0 most of the time), the ambient component (does not take vectors into account), the diffuse component (that takes into account the normal and the light vectors) and the specular component

(that takes into account all vectors).

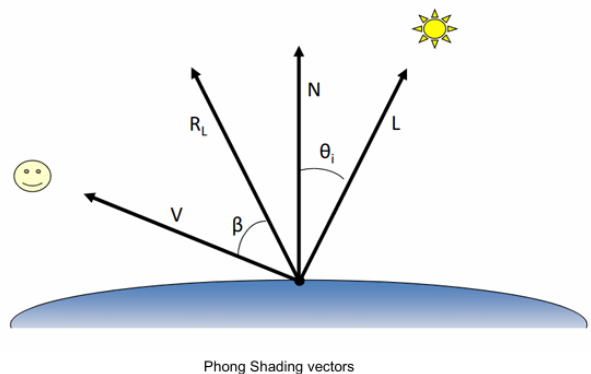


Figure 1. Phong Shading taken from the class notes

With these components in mind, the Phong Shading model produces a result with the following formula:

$$K_e + K_a I_a + K_d I_d (N \cdot L) + K_s I_s (R \cdot V)^s$$

Where the K refers to the object properties and the I to the light properties. The value s refers to the *shininess* of the material, that controls specular reflections.

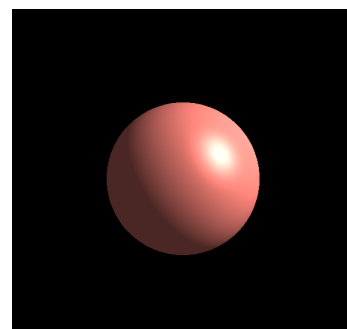


Figure 2. Our Phong Shading results

2.2. Environment Maps and Textures

In this project we also used different textures to store different properties of the materials such as the color, the metalness or the roughness. Metalness and roughness textures will be relevant for the PBR part, but for now we can combine color textures with phong shading.

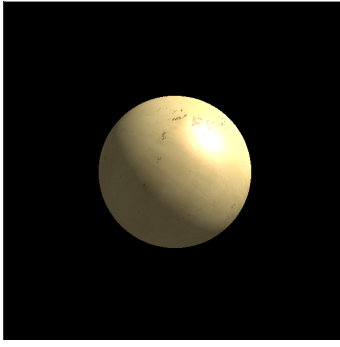


Figure 3. Phong Shading with a texture as a color

For this project we will also need to use environment maps. Unlike usual 2D textures that cover the surface of an object, environment maps are textures that cover the whole environment. In this project, we used them to generate skyboxes to represent the environment of the scene, to generate reflections, and to store the environment lighting.

2.3. Reflections

Another relevant part of our project is the reflection computation. If we reflect the view vector with respect to the normal vector of the surface, we can then use that same reflected vector to access the environment map that surrounds the whole scene. This way, we can easily simulate mirror-like reflections of the environment on a given material.



Figure 4. Result of the reflection of the environment maps

2.4. Simple Physically Based Rendering method

The goal of creating a Physically Based renderer is to incorporate the roughness and metalness of a material to our illumination computation and to take into account more detailed light-surface interactions.

The end goal of the various realistic illumination techniques used for rendering is to approximate the *Rendering Equation* which exactly describes how the behavior of the light, but cannot be exactly computed in real time:

$$L_o(p, \omega_o) = \int_{\Omega} f_r(p, \omega_i, \omega_o) L_i(p, \omega_i) n \cdot \omega_i d\omega_i$$

This equation computes the amount of light that leaves the object towards the observer (outgoing direction) taking into account each possible direction for the light, and the properties of the material that modify the amount of outgoing light. Luckily, our simple PBR model only renders one single

isolated object with a single point light, so we only have to take into account one possible light direction. The goal of this part of the project will then be to implement the BRDF function f_r , that approximates how much each incoming light contributes to the final reflected light given the material properties.

For this project we are going to use the Cook-Torrance BRDF, which divides the function in two parts: diffuse and specular.

$$f_r = k_d f_{\text{lambert}} + k_s f_{\text{cook-torrance}}$$

Where k_d and k_s represent the ratio of incoming light that is refracted (k_s) and reflected (k_d). The lambert part is really easy to compute:

$$f_{\text{lambert}} = \frac{c}{\pi}$$

With c being the color (also albedo) of the material. Take into account that the result of the function will then be multiplied by $n \cdot \omega_i$ which is equivalent to $N \cdot L$ in our phong model, making the diffuse part of our BRDF really similar to the diffuse component of the phong model. There are other diffuse functions, but using lambert is a good approximation. The specular part, however, is a bit more complex:

$$f_{\text{CookTorrance}} = \frac{FGD}{4(\omega_o \cdot n)(\omega_i \cdot n)}$$

The numerator of this function is the product of three different functions that depend on the properties of the material:

- **Fresnel Equation (F):** Describes the ratio of surface reflection at different angles
- **Geometry Function (G):** Describe the self-shadowing of the microfacets. In rough materials the microfacets can overshadow other microfacets reducing the light that the surface reflects.
- **Normal Distribution Function (D):** This function approximates the reflection effect of the microfacets of the material. It takes into account the number of microfacets that are aligned with the normal. This function depends on the roughness of the material.

The microfacets are the little microscopic imperfections of the surface that produce multiple reflections and interactions with the light. We cannot model every single microscopic imperfection of the object so we use the previously mentioned functions to describe how the light interacts with those microfacets. The rougher a surface is, the more imperfections it will have, so rougher surfaces will produce blurrier reflections.

There are multiple functions that can be used as our Fresnel Equation, Geometry Function or Normal Distribution Function. For the Geometry Function and Normal Distribution Function we tried and implemented multiple functions taken from the Jordan Stevens blog about Physically Based Rendering[2]. For the Fresnel Equation we used the Schlick approximation:

$$F_{\text{Schlick}}(F_0, l, h) = F_0 + (1 - F_0)(1 - (l \cdot h))^5$$

Where F is the *Fresnel* of the material (it is provided as a parameter of a given material), l is the light vector and h is the *halfway* vector, which is computed as the average of the light and the view vectors.

Finally, the ratios k_d and k_s depend on the metalness of the material. for k_s we can take the result of the Fresnel Equation and then we can set k_d to $1 - k_s$. We can further attenuate k_d setting $k_d \leftarrow k_d * (1 - \text{metalness})$ as the more metallic a material is, the less important the diffuse term becomes.

In summary, our initial equation becomes the following one:

$$L_o(p, \omega_o) = \int_{\Omega} k_d \frac{c}{\pi} + \frac{FGD}{4(\omega_o \cdot n)(\omega_i \cdot n)} L_i(p, \omega_i) n \cdot \omega_i d\omega_i$$

We have to take into account that we do not have to multiply by k_s as $k_s = F$, and in our application we will only have one single light (so we can skip the integration and just compute the term inside the integral once).

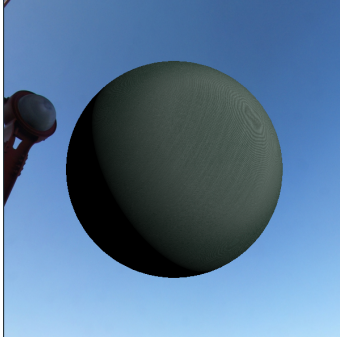


Figure 5. Rough Ball rendered using the simple PBR method

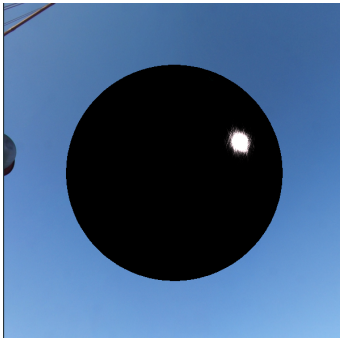


Figure 6. Metallic Ball rendered using the simple PBR method. There is only one point light so the ball can not reflect the environment

2.5. Physicall Based Rendering with IBL

In the previous section we assumed that we had only one point light (and maybe we can extend that to a finite number of point lights summing them all), however, in most scenes we will have light coming from every direction. If we used point lights to model the environment light it would be really unsustainable. For this reason we can use environment maps to store the incoming light from every direction. We can even go a step further and, as we are already using environment maps, store parts of the integral in those environment maps

to accelerate our computations. It is important to note that, for this method, we have to consider the whole scene as a single point (because the incoming light will only depend on the direction and not the position of the surface), which is a reasonable approximation if we consider the environment light to come from really far away.

Dividing our function into its specular and diffuse components, we can easily see that the diffuse part does not depend on the view direction and it only depends on the direction of the light:

$$k_d \frac{c}{\pi} \int_{\Omega} L_i(\omega_i) n \cdot \omega_i d\omega_i$$

This way, we can use an environment map to store, for every normal direction, the diffuse lighting that is received from that direction. This means precomputing the integral at every possible direction and storing that precomputation in a cube map. Finally we can render different materials sampling the cube map at the normal direction of every point of the object and then multiplying that value by c and k_d . Logically, we have no way of computing the whole integral, so we need to use approximations and discretize it. The discretization that we used is the following (using spherical coordinates for the direction):

$$L_o(p, \phi_o, \theta_o) = k_d \frac{c\pi}{n_1 n_2} \sum_{\phi=0}^{n_1} \sum_{\theta=0}^{n_2} L_i(p, \phi_i, \theta_i) \cos(\theta) \sin(\theta)$$

Sadly, the specular part also depends on the view direction, and we can not precompute a cube map for every possible view direction as that would have an enormous cost. Still, we can rely on the following approximation that allows for some precomputation:

$$k_s \int_{\Omega} \frac{FGD}{4(n \cdot \omega_i)(n \cdot \omega_o)} L_i(\omega_i) n \cdot \omega_i d\omega_i =$$

$$k_s \frac{FG}{4(n \cdot \omega_i)(n \cdot \omega_o)} \int_{\Omega} D(n \cdot \omega_i) d\omega_i$$

This formula separates the computation in two parts that can be computed sparately.

The integral of the Normal Distribution Function controls the reflection of the environment, so it has to take into account every possible incident direction of the light. Luckily, the view vector will only affect the direction of the reflection, so we can still store all the information in a cube map and compute the integral the same way we computed reflections in a previous section. However, the reflection has to take into account the roughness of the material, the rougher the material the blurrier the reflection. This problem can easily be solved creating different maps for different roughness values and then use those maps as LODs of the texture, so we will combine those precomputed textures depending on the roughness of the material to render our object.

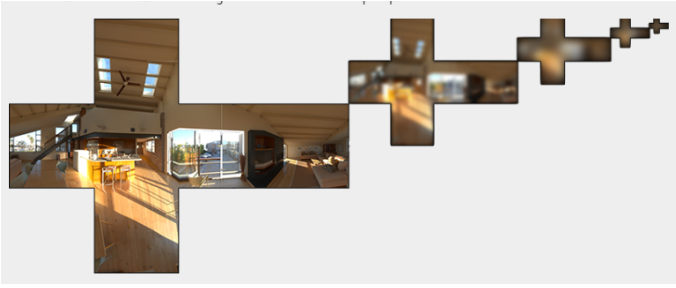


Figure 7. Different LODs of the same texture. Higher mip equals higher roughness

Finally, to render an object we will compute the reflection of the view vector, use the reflection to sample the cube map (taking into account the roughness to access the LODs) and then we will multiply that value taken from the cubemap by $k_s \frac{FG}{4(n \cdot \omega_i)(n \cdot \omega_o)}$ to obtain the specular component. After that, we will only have to sum the specular and diffuse parts to obtain the resulting lighting.



Figure 8. Result of the IBL PBR method with a rough and metallic object

2.6. Gamma Correction

Finally, we also added gamma correction to our shaders. Our eyes do not perceive light linearly and have higher resolution for darker colors, but the lighting on the computer is computed linearly. For that reason most modern monitors adjust the lighting, raising it to the power of a parameter *gamma*, to make it non-linear and match how our eyes work.

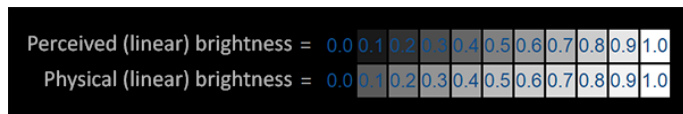


Figure 9. Our eyes do not see the light linearly

This may not be ideal for our rendering program as we are treating colors in a linear way for our computations. For this reason we can raise our output colors to the power of $1/\gamma$ to compensate the correction done by the monitor. The typical gamma of modern monitors is 2.2, so that is the value that we are going to use (although we can test more values to see the difference).

3. Shader Implementation

3.1. Texture Shader

Our application allows loading multiple textures (for color, metalness and roughness) and visualizing them with a simple

shader without any lighting.

The vertex shader just outputs the texture coordinates to the fragment shader and computes the position as usual, using the projection, view and model matrices.

```
1 #version 330
2
3 layout (location = 0) in vec3 vert;
4 layout (location = 1) in vec3 normal;
5 layout (location = 2) in vec2 texCoord;
6
7
8 uniform mat4 projection;
9 uniform mat4 view;
10 uniform mat4 model;
11
12 out vec2 vTexCoords;
13
14
15 void main(void) {
16     vTexCoords = texCoord;
17     gl_Position = projection*view*model*vec4(
18         vert,1.0);
19 }
```

Code 1. texture vertex shader

The fragment shader is also really simple, the code provides the selected texture (color, metalness or roughness) as a uniform and the shader uses that texture for the output.

```
1 #version 330
2
3 out vec4 frag_color;
4 uniform sampler2D current_texture;
5 in vec2 vTexCoords;
6
7 void main (void) {
8     frag_color = texture(current_texture,
9         vTexCoords);
10 }
```

Code 2. texture fragment shader

3.2. Phong Shader

Our implementation of the phong shading uses coordinates in the view space, so we need to give the fragment shader the information of the multiple parameters transformed into view space. The vertex shader just transforms the different relevant parameters to view space and outputs them to the fragment shader:

```
1 void main(void) {
2     normal_vec = normal_matrix * normal;
3     view_vec = -(view*model*vec4(vert,1.0)).xyz;
4     light_vec = (view*vec4(light,1.0)).xyz +
5         view_vec;
6
6     gl_Position = projection*view*model*vec4(
7         vert,1.0);
8 }
```

Code 3. Phong vertex shader (shortened)

Then, the fragment shader performs the computation of the phong shading model. It is important to note that there is a define at the start of the code. That define controls whether the color of the material (which will be the k_a and k_d in this case) is set to a solid color or is given by a texture. If the define line is commented that means that the shader uses a solid color, it uses a texture otherwise.

```

1 #version 330
2 // #define MAPS
3 #ifdef MAPS
4     uniform sampler2D color_map;
5 #else
6     uniform vec4 matAmbient = vec4(0.97, 0.51,
7                                     0.47, 1.0);
8     uniform vec4 matDiffuse = vec4(0.97, 0.51,
9                                     0.47, 1.0);
10 #endif
11
12 // Declaration of the rest of uniforms...
13
14 vec4 phong(vec3 N, vec3 V, vec3 L) {
15     N = normalize(N); V = normalize(V); L =
16     normalize(L);
17     vec3 R = reflect(-L, N);
18     float NdotL = max(0.0, dot(N, L));
19     float RdotV = max(0.0, dot(R, V));
20
21     float Idiff = NdotL;
22     float Ispec = 0;
23     if (NdotL > 0) Ispec = pow(RdotV, shininess);
24
25     #ifdef MAPS
26     vec4 albedoAmbient = texture(color_map,
27                                 vTexCoords);
28     vec4 albedoDiffuse = albedoAmbient;
29     #else
30     vec4 albedoAmbient = matAmbient;
31     vec4 albedoDiffuse = matDiffuse;
32     #endif
33
34     vec4 res = albedoAmbient * lightAmbient +
35               albedoDiffuse * lightDiffuse * Idiff +
36               matSpecular * lightSpecular * Ispec;
37     return res;
38 }
39
40 void main (void) {
41     frag_color = phong(normal_vec, view_vec,
42                        light_vec);
43 }

```

Code 4. Phong fragment shader (shortened)

Regarding the phong section, it does not have anything special, it starts computing each necessary value (every dot product and the reflected vector) and then it combines every value (the final computation is done in line 31).

3.3. Sky Shader

We also need a shader to render the sky, which will be rendered in the background. To render the sky we will render a cube using a cube map to texturize it. The cube has to always appear really far away, so it should not take into account translations of the camera so, for this reason, the view matrix that is passed as a uniform to the shader is transformed using this line of code:

```
glm::mat4x4 skyView = glm::mat4x4(glm::mat3x3(view));
```

Which extracts just the rotation of the camera

```

1 #version 330
2
3 layout (location = 0) in vec3 vert;
4 layout (location = 1) in vec2 texCoord;
5
6 out vec3 TexCoords;
7

```

```

8 uniform mat4 projection;
9 uniform mat4 view;
10 uniform mat4 model;
11
12 void main(void) {
13     TexCoords = vert;
14     vec4 temp = projection*view*vec4(vert, 1.0);
15     gl_Position = temp;
16 }

```

Code 5. Sky vertex shader

The fragment shader just needs to sample the cube map at the given texture coordinate.

```

1 #version 330
2
3 out vec4 frag_color;
4 in vec3 TexCoords;
5 uniform samplerCube specular_map;
6
7 void main (void) {
8     frag_color = texture(specular_map, TexCoords);
9 }

```

Code 6. Sky fragment shader

3.4. Reflection Shader

Our reflection will be done in model space, so the vertex shader will have to output the normal and position of the vertex in model space.

```

1 #version 330
2
3 layout (location = 0) in vec3 vert;
4 layout (location = 1) in vec3 normal;
5 layout (location = 2) in vec2 texCoord;
6
7 uniform mat4 projection;
8 uniform mat4 view;
9 uniform mat4 model;
10
11 uniform mat3 normal_matrix;
12
13 out vec3 world_normal;
14 out vec3 world_pos;
15
16 void main(void) {
17     world_normal = mat3(transpose(inverse(model)
18                                ))*normal;
19     world_pos = vec3(model*vec4(vert, 1.0));
20     gl_Position = projection * view * model*
21                   vec4(vert, 1.0);
22 }

```

Code 7. Reflection vertex shader (shortened)

Then, the fragment shader will compute the view vector, reflect it with respect to the normal, and use the reflected vector to sample the cube map. This fragment shader also includes the possibility of changing the roughness, to change the LODs of the texture and test the reflection part of the IBL PBR. When roughness is equal to 0 we are in the perfect reflection situation.

```

1 #version 330
2 // Change this value to change the reflection
3 // This can be used to test a part of the IBL
4 // shader
5 float roughness_lod = 0.0;
6 in vec3 world_normal;

```

```

7 in vec3 world_pos;
8
9 void main (void) {
10     vec3 cameraPos = vec3(inverse(view) * vec4
11         (0.0,0.0,0.0,1.0));
12
13     vec3 incident = normalize(world_pos -
14         cameraPos);
15     vec3 reflected = reflect(incident,normalize(
16         world_normal));
17
18     frag_color = textureLod(prefilter_map,
19         reflected,roughness_lod);
20 }

```

Code 8. Reflection fragment shader (shortened)

3.5. Simple PBR Shader

Our implementation is basically what we talked about in previous sections. We first compute necessary values for the final computation: the halfway vector h , the new fresnel value taking into account the metalness of the material to simulate the way the reflection is tinted in metallic objects (the higher the metalness the closer the reflection is to the albedo of the material), and, finally all the relevant dot products. After the initial computation we compute F, G and D . The computation of F is done using the $F_{Schlick}$ approximation mentioned earlier while G and D use different functions taken from the Jordan Stevens blog[2] (there is a global parameter that controls which function is used). Finally we compute the diffuse component, we modify the ratio k_d according to the metalness and we compute the final result. It is important to note that the result has to be scaled by $n \cdot \omega_i$ (which is `ndotl` in our code) as it is not done at any point of the computation.

```

1 vec3 pbs(vec3 l, vec3 v, vec3 n, float met,
2     float rough, vec3 albedo) {
3     n = normalize(n); v = normalize(v); l =
4     normalize(l);
5     vec3 h = normalize(l + v);
6     vec3 F0 = mix(fresnel, albedo, met);
7
8     float ndotl = max(0.0000001, dot(n,l));
9     float ndotv = max(0.0000001, dot(n,v));
10    float ldotv = max(0.000000, dot(l,v));
11
12    vec3 F = FresnelReflectanceFunction(F0,l,h);
13    float G = GeometryFunction(l,v,h,n);
14    float D = MicroGeometryDistribution(h,n,
15        rough);
16    vec3 fs = (F*G*D)/(4 * ndotl * ndotv);
17
18    vec3 fd = Flambert(albedo);
19
20    vec3 ks = F;
21    vec3 kd = vec3(1) - ks;
22    kd *= 1 - met;
23
24    vec3 res = kd*fd + fs;
25    res *= ndotl;
26    return res;
27 }

```

Code 9. PBR shader function (shortened)

The vertex shader is the same as the phong shading vertex shader and the fragment shader calls the function `pbs` with the parameters obtained from the vertex shader (light direction, view direction and normal direction) and the properties of the material (metalness, roughness and albedo). The shader also

includes a *define* to switch between using global values for the roughness, albedo and metalness and using textures.

Additionally, we also included gamma correction in our shader. The main function of the fragment shader calls the `pbs` function and then it scales the result to apply the gamma correction based on the value of the gamma (which is a global variable).

```

1 void main (void) {
2     vec3 light = pbs(light_vec,view_vec,
3         normal_vec, met, rough, albedo);
4     frag_color = vec4(light,1.0);
5     frag_color.rgb = pow(frag_color.rgb, vec3
6         (1.0/gamma));
7 }

```

Code 10. gamma correction of our shader

3.6. IBL PBR Shader

This shader is very similar to our previous PBR shader with a few key differences: First, we do not have an incoming light direction as we consider the light direction to be the same as the view direction (we set $l = v$ before the function), this also makes the computation of the halfway vector different. For the halfway vector we used $normalize(l + v)$ (same as before) and $normalize(l + n)$ and both give good results (although different); Then, the Fresnel equation uses the normal instead of the halfway vector, as suggested by openGL; and, finally the computation of the diffuse term and the NDF function are done accessing the precomputed environment maps. The diffuse irradiance map also takes into account the product $n \cdot \omega_i$ so we only have to multiply the specular term by that factor (although multiplying the whole result by that dot product tends to produce a really similar result).

```

1 vec3 ibl_pbs(vec3 l, vec3 v, vec3 n, float met,
2     float rough, vec3 albedo) {
3     n = normalize(n); v = normalize(v); l =
4     normalize(l);
5
6     //The halfway vector does not have much
7     //sense in this situation (with ambient light)
8     //Still we are going to consider that light
9     //comes from the normal direction.
10    vec3 h = normalize(l + n);
11
12    vec3 F0 = mix(fresnel, albedo, met);
13
14    float ndotl = max(0.0000001, dot(n,l));
15    float ndotv = max(0.0000001, dot(n,v));
16    float ldotv = max(0.0000001, dot(l,v));
17
18    //Learn OpenGL suggests using the angle
19    //between l and n to compute the F_Schlick
20    //function
21    vec3 F = FresnelReflectanceFunction(F0,l,n);
22    float G = GeometryFunction(l,v,h,n);
23
24    vec3 cameraPos = vec3(inverse(view) * vec4
25        (0.0,0.0,0.0,1.0));
26    vec3 incident = normalize(world_pos -
27        cameraPos);
28    vec3 reflected = reflect(incident,normalize(
29        world_normal));
30
31    float roughness_lod = rough*5.0;
32    vec3 D = textureLod(prefilter_map,reflected,
33        roughness_lod).rgb;
34 }

```

```

24  vec3 fs = (F*G*D)/(4 * ndotv * ndotl);
25
26  vec3 diff_irradiance = texture(diffuse_map,
27  normalize(world_normal)).rgb;
28  vec3 diffuse = albedo * diff_irradiance;
29
30  vec3 ks = F;
31  vec3 kd = vec3(1) - ks;
32  kd *= 1 - met;
33
34  vec3 res = kd*diffuse + fs*ndotl;
35  //res *= ndotl;
36  //res *= diffuse;
37  return res;
38 }

```

Code 11. IBL PBR shader function (shortened)

The vertex shader only has the additional job of sending the required vectors for the reflection the the fragment shader and the fragment shader main function is the same as before (with the difference that it calls the function using the view direction as the light direction).

3.7. Diffuse Irradiance Texture generation

To generate the diffuse irradiance maps we created a button that takes the current specular map/skybox, computes the proper convolution and then stores that convolution in a folder which can be then loaded. To compute the convolution, we created pair of shaders (vertex and fragment) that take one face of the cube map and compute the convolution in the following way (We are just showing the fragment shader because the vertex one is really straight forward).

The fragment shader first generates a coordinate system with the direction that we are using to generate the convolution as the z direction (lines 12-14). Then it generates multiple points on a hemisphere (for loops of lines 19 and 20) and, for each point, it converts the point to world coordinates using the previously mentioned coordinate system and then it samples the specular map at those world coordinates. The final result is the sum of all samples divided over the number of samples. The idea behind this shader was taken from LearnOpenGL[3].

```

1  vec3 point_on_sphere(float u, float v, float r)
2  {
3      float x = r*sin(u)*cos(v);
4      float y = r*sin(u)*sin(v);
5      float z = r*cos(u);
6      return vec3(x,y,z);
7  }
8
9  void main (void) {
10     //We can take the vertex coordinates as the
11     normal vector
12     vec3 norm = normalize(vTexCoords);
13
14     vec3 irradiance = vec3(0.0);
15     vec3 up_vec = vec3(0.0,1.0,0.0);
16     vec3 right_vec = normalize(cross(up_vec,
17     norm));
18     up_vec = normalize(cross(norm,right_vec));
19
20     float sampleDelta = 0.025;
21     int nSamples = 0;
22     for (float phi = 0.0; phi < 2.0* PI; phi +=
23     sampleDelta)
24         for (float theta = 0.0; theta < 0.5*PI;
25         theta += sampleDelta) {

```

```

21     vec3 sample = point_on_sphere(theta,
22     phi, 1);
23
24     vec3 world_sample = sample.x *
25     right_vec + sample.y * up_vec + sample.z*
26     norm;
27
28     irradiance += texture(specular_map,
29     world_sample).rgb*cos(theta)*sin(theta);
30     ++nSamples;
31 }
32
33 irradiance = (PI * irradiance)/(float(
34     nSamples));
35 frag_color = vec4(irradiance,1.0);
36 }

```

Code 12. Diffuse Map generation (shortened)

3.8. Prefiltered Maps for IBL

Prefiltered maps are computed when loading the specular map and they are not stored, as that would imply storing a lot of images, and it would also imply modifying the code to load different LODs when loading a cube map.

The computation of these prefiltered maps has been done like with the diffuse irradiance maps. We used a shader that renders a face of the cube map and then performs all necessary computations on that face. The fragment shader is really long and we used an implementation of this prefiltering explained on the website LearnOpenGL[3], so we are going to explain the reasoning behind that technique in this section instead of showing the whole code.

To simulate reflection we have to take into account that the further away a ray is from the reflected direction, the weaker its contribution to the total light, that contribution becomes 0 when we sample rays outside a certain *lobe*. The rougher the material the bigger the *lobe*.

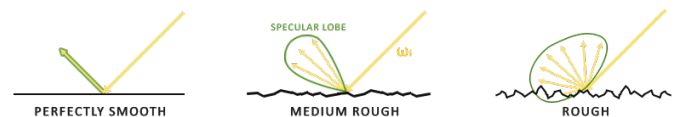


Figure 10. The smoother a material is the weaker the smaller the reflection lobe becomes

To integrate all the light taking this into account we could use the same technique used for the diffuse irradiance map and add a scale factor that downscales the contribution of rays depending on the distance with the reflection direction. This would, in theory, work, but there are better methods to compute the integral.

Instead of using the previous method we can use a Monte Carlo technique, that consists on taking random samples and averaging the result to compute the integral. Or we can go even further and use **Importance Sampling** which is based on the Monte Carlo method but it uses a biased method to take samples, making it more probable to sample elements that will have a lot of importance on the final computation.

The main part of the fragment shader that implements this importance sampling is the following:

```

1 void main (void) {
2     vec3 N = normalize(vTexCoords);
3     vec3 R = N;
4     vec3 V = R;
5
6     const uint SAMPLE_COUNT = 1024u;
7     float totalWeight = 0.0;
8     vec3 prefilteredColor = vec3(0.0);
9     for(uint i = 0u; i < SAMPLE_COUNT; ++i)
10    {
11        vec2 Xi = Hammersley(i, SAMPLE_COUNT);
12        vec3 H = ImportanceSampleGGX(Xi, N,
13        roughness);
14        vec3 L = normalize(2.0 * dot(V, S) * S -
15        V);
16
17        float NdotL = max(dot(N, L), 0.0);
18        if(NdotL > 0.0)
19        {
20            prefilteredColor += texture(
21            specular_map, L).rgb * NdotL;
22            totalWeight += NdotL;
23        }
24    }
25    prefilteredColor = prefilteredColor /
26    totalWeight;
27
28    frag_color = vec4(prefilteredColor, 1.0);
29 }

```

Code 13. Prefiltered specular generation (shortened)

As you can see, it consists on a loop that takes multiple samples, and adds up all those samples scaled by their weight (the weight being the angle between the given direction and the sample direction or $N \cdot L$ being N the direction that we are checking). lines 11-13 are used to obtain the random sample, biased to be around the expected microsurface's halfway vector. The functions used to generate the sample have been taken from LearnOpenGL[3], as mentioned earlier.

4. Discussion

In this project we successfully created a Physically Based renderer that computes the illumination of a scene taking into account different properties of the material. However, the result of this project is not perfect and can be extended in multiple ways: First, we considered simple scenes with a single object and considering the whole object as a single point for lighting computations; We also made a lot of approximations and simplifications to accelerate the rendering, we could even try different (and possibly more exact approximations) to produce even better results; and we also did not implement the BRDF integration map, which could have improved the performance of our shaders.

Additionally, the biggest problem of this project is that resulting code (specially the c++ code for the glWidget) is really messy. The initial idea was to add an option to store the precomputed maps in the IBL part to load them whenever we wanted and, therefore, save even more time. This idea of storing the textures on the disk produced a lot of problems and required a lot of modifications to the basic skeleton that was initially provided. All this modifications coupled by a lack of time (the precomputations of the different maps were left for the end and we did not expect all the problems that we had) made the code a little bit disorganized and even incomplete (Storing diffuse irradiance maps is implemented,

but the prefiltered map for the specular part cannot be saved on disk).

We also found a problem where the viewport that contains the rendering is smaller than usual, leaving a blank section on the application, this has to do with a problem with the skeleton and that problem was not really addressed during this project. The program is still usable and this problem only has effect on the dimensions of the viewport (and it also appears only on some displays).

Despite of all of its limitations, this project is still operative and it provides a great start to more realistic lighting and a huge improvement from the simple Phong shading.

5. Acknowledgments

It is important to acknowledge that this project was mainly conducted in the context of a master's degree project and it was done with the help of multiple resources: The class notes of the FRR course provided by the university[1], the Learn OpenGL website[3] and a website made by Jordan Stevens[2] about Physically Based Rendering. Without the help of all those different resources this project will have been impossible to made. Additionally, all the code is based on a skeleton made by Imanol Munoz-Pandilella (which is also based on a previous skeleton made by Marc Comino).

References

- [1] FIB, *Fast realistic rendering*, Accessed: 2025-04-28, 2025. [Online]. Available: <https://www.fib.upc.edu/en/studies/masters/master-innovation-and-research-informatics/curriculum/syllabus/FRR-MIRI>
- [2] J. Stevens, *Physically based rendering algorithms: A comprehensive study in unity3d*, Accessed: 2025-04-28, 2025. [Online]. Available: <https://www.jordanstevensstechart.com/physically-based-rendering>
- [3] J. D. Vries, *Learnopengl*, Accessed: 2025-04-28, 2025. [Online]. Available: <https://learnopengl.com/>